0ef05b1c-80ee-40bf-a96b-25e18637c2fd

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Alfanous-team/alfanous' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/alfanous/alfanous (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T19:29:52.302Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add Arabizi transliteration with multi-candidate Arabic output and Quran word filtering
</pr_title>

<pr_description>
Arabizi (Arabic chat alphabet) is inherently ambiguous — digraphs like `sh` can map to ش or to ه+س — so a single-string output is insufficient. This adds Arabizi support that returns all possible Arabic interpretations, and integrates it into the QSE search pipeline as a replacement for Buckwalter.

Changes

`src/alfanous/romanization.py`
- **`ARABIZI2UNICODE`**: Single-character map covering digit-based sounds (`3→ص`, `9→ع`, `7→ح`, etc.) and common Latin letters (`s→س`, `h→ه`, `k→ك`, …). Fixed `e→ي` (vowel sound, not ع) and added `u→و`.
- **`ARABIZI_DIGRAPHS`**: Two-character sequences mapping to a single Arabic letter: `sh→ش`, `th→ث`, `kh→خ`, `dh→ذ`, `gh→غ`, `eh→ة` (ta marbuta for terminal feminine suffix).
- **`arabizi_to_arabic_list(string, ignore="")`**: Recursively explores all segmentations returning a deduplicated list of candidate Arabic strings, with enhanced rules:
  - **Rule A** (precedence): Multi-character codes processed before single characters.
  - **Rule B** (Al- prefix): Word-initial `el` or `al` generates a `ال`-prefixed candidate. Spaces reset the word-start flag so every word is handled independently.
  - **Rule C** (gemination): Doubled non-vowel consonants (e.g. `ll`, `tt`) generate an additional candidate with the letter followed by shadda (ّ). Vowels are excluded; digraph units are documented as distinct.
  - **Rule D** (vowel mapping): Initial `a` also yields `أ` (hamza-on-alef); terminal `a` also yields `ى` (alef maqsura); digraph `eh` yields `ة` (ta marbuta).
  - **Short vowel omission**: Every vowel (`a`, `e`, `i`, `o`, `u`) also yields an empty-string candidate, since short vowels are not written in unvocalized Arabic script (e.g. `salameh` → includes `سلامة`; `a5barak` → includes `أخبارك`, `اخبارك`).
- **`filter_candidates_by_wordset(candidates, wordset)`**: Shared utility that filters a candidate list to entries where every space-separated token exists in a given word set, falling back to all candidates if none match.

`src/alfanous/paths.py`
- Added `WORD_FILE` path constant pointing to `word.json`.

`src/alfanous/data.py`
- Added **`quran_unvocalized_words()`**: a `@lru_cache`-backed function that loads `word.json` and returns a `frozenset` of the 14 870 unvocalized Quranic word forms (`word_` field). Falls back to an empty frozenset on I/O errors so callers degrade gracefully.

`src/alfanous/outputs.py`
- **Replaced Buckwalter with Arabizi for QSE**: When a query contains no Arabic characters, it is now treated as Arabizi. Candidates are generated with `arabizi_to_arabic_list()` and then filtered through `filter_candidates_by_wordset()` against

`quran_unvocalized_words()`. If no candidates match the Quranic word list (e.g. dialect words), the full unfiltered candidate set is used as fallback. Arabic queries and field-qualified queries (containing `:`) pass through unchanged.

`src/tests/test_engines.py`
- Replaced `test_advanced_search_buckwalter` with `test_advanced_search_arabizi`.
- Extended `test_arabizi_transliteration()` to cover short vowel omission (asserting `أخبارك` ,`اخبارك` ,`سلامة` appear in candidates).
- Added `test_arabizi_quran_word_filter()` using the shared `filter_candidates_by_wordset` utility, covering: Quran match (`يعطيك` from `ya36eek`), ta-marbuta match (`بكرة` from `bokreh`), and graceful fallback for non-Quranic dialect words.

Example

```python
from alfanous.romanization import arabizi_to_arabic_list, filter_candidates_by_wordset

arabizi_to_arabic_list("sh")        # ['سه' ,'ش']           — digraph vs. two letters
arabizi_to_arabic_list("el 7al")    # ['يل حال' ,'ال حال']  — Rule B: el → ال
arabizi_to_arabic_list("ll")        # ['لل' ,'لّ']          — Rule C: gemination
arabizi_to_arabic_list("salameh")   # [..., 'سلامة', ...]    — short vowel omission + eh → ة
arabizi_to_arabic_list("a5barak")   # [..., 'اخبارك', 'أخبارك', ...] — short vowel omission
arabizi_to_arabic_list("3la")       # ['على' ,'علا']        — Rule D: terminal a → ى
arabizi_to_arabic_list("k*b", ignore="*")  # ['ك*ب']         — operator passthrough

from alfanous.data import quran_unvocalized_words
filter_candidates_by_wordset(arabizi_to_arabic_list("ya36eek"), quran_unvocalized_words())
# → ['يعطيك']  — filtered to the single Quranic match
```

<issue_title>Add Arabizi transliteration, the same we do for buckwalter, it should generate a list of potential arabic words</issue_title>
<issue_description></issue_description>

>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

ة
...

Done in `0aa0cac`:

Short vowel omission — every vowel (`a`, `e`, `i`, `o`, `u`) now also generates an empty-string candidate, matching unvocalized Arabic where short vowels are not written:
- `arabizi_to_arabic_list("salameh")` → includes `''سلامة''`
- `arabizi_to_arabic_list("a5barak")` → includes `''اخبارك''`, `''أخبارك''`

Quran word filtering — added `quran_unvocalized_words()` in `data.py` (loads `word.json`, returns a cached frozenset of 14 870 unvocalized words) and `filter_candidates_by_wordset()` in `romanization.py` as the shared filtering utility. `outputs.py`

(QSE) now filters each Arabizi query's candidates to those where every token is a known Quranic word, falling back to all candidates when none match (e.g. dialect words not in the Quran).

Example: `ya36eek` → filtered to `['يعطيك']`; `salameh` → no Quran match, falls back to all candidates.
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6 change test to "el 7md" == "الحمد"

>   assert arabizi_search("7md") > 0   # حمد (hamd/ praise)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E   AssertionError: assert 0 > 0
E   + where 0 = </comment_old>

<comment_new>
<comment_id>3993081540</comment_id>
<author>@assem-ch</author>
@copilot try again

</comment_new>

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.

* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

The fix is simple and targeted: the test `arabizi_search("7md")` was asserting `> 0` results but returning 0 because "حمد" alone doesn't appear as an unvocalized Quranic word in the index. Changing the input to `"el 7md"` produces the candidate "الحمد" (al-hamd), which does appear in the Quran (e.g., Surah Al-Fatiha: "الحمد لله رب العالمين"), so the search now returns results correctly.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Alfanous-team/alfanous' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/alfanous/alfanous (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T22:45:39.624Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add Arabizi transliteration with multi-candidate Arabic output and Quran word filtering
</pr_title>

<pr_description>
- [x] Replace Buckwalter with Arabizi for QSE in outputs.py
- [x] Implement arabizi_to_arabic_list() with Rules A-D and short vowel omission
- [x] Add filter_candidates_by_wordset() utility
- [x] Add quran_unvocalized_words() in data.py
- [x] Add WORD_FILE path in paths.py
- [x] Update tests: test_advanced_search_arabizi, test_arabizi_transliteration, test_arabizi_quran_word_filter
- [x] Fix failing test: change `arabizi_search("7md")` → `arabizi_search("el 7md")` (الحمد is in the Quran; حمد alone returns 0 results)

<issue_title>Add Arabizi transliteration, the same we do for buckwalter, it should generate a list of potential arabic words</issue_title>
><issue_description></issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6 if the query is latin, try search with Arabizi for QSE, replace Buckwalter
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6 if the query is latin, try search with Arabizi for QSE, replace Buckwalter

Done in d746d8f. In `outputs.py`, the Buckwalter transliteration has been replaced with Arabizi for QSE. When the query contains no Arabic characters, `arabizi_to_arabic_list()` generates all candidate Arabic strings (handling digraph ambiguity like `sh` → [ش, سه]) and joins them with spaces for OR semantics in the Whoosh OrGroup parser. Arabic queries pass through unchanged.
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6
| Arabizi Input | Target Arabic | Meaning
-- | -- | -- | --
1 | Keefak? | كيفك؟ | How are you? (Levant)
2 | Mnoo7 | منوح | Fine/Good
3 | Shlonak? | شلونك؟ | How are you? (Gulf/Iraq)
4 | Waynak? | وينك؟ | Where are you?
5 | Ma3 el salameh | مع السلامة | Goodbye
6 | 7abibi | حبيبي | My love
7 | Kil saneh w enta salem | كل سنة و انت سالم | Happy New Year/ Birthday
8 | Ya3teek el 3afye | يعطيك العافية | May God give you health
9 | Sho a5barak? | شو أخبارك؟ | What’s your news?
10 | Ma fi moshkeleh | ما في مشكلة | No problem
11 | Yalla nemshi | يلا نمشي | Let's go/walk
12 | Bokra nshoofak | بكرة نشوفك | See you tomorrow
13 | Tasharrafna | تشرفنا | Nice to meet you
14 | Inshallah 5air | إن شاء الله خير | Hopefully all is good
15 | Wallah ma ba3ref | والله ما بعرف | I swear I don't know
16 | Ktir 7ilo | كتير حلو | Very beautiful
17 | 3ala rasi | على راسي | My pleasure (on my head)
18 | Yislamoo | يسلمو | Thank you (may your hands be safe)
19 | Shu 3am ta3mel? | شو عم تعمل؟ | What are you doing?
20 | Mashi el 7al | ماشي الحال | It's going okay
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6

The Enhanced Spec (Logic Rules)
Rule A (Precedence): Process multi-character codes (3', kh, sh) before single characters.
Rule B (The "Al-" Prefix): If a word starts with el- or al-, it must map to the Arabic definite article ال.
Rule C (Gemination): Doubled Latin consonants (e.g., bb, tt) should be mapped to a single Arabic letter with a Shadda (ّ).
Rule D (Vowel Mapping):
Initial a or 2

 أ
Terminal a or eh

 ة or ى (depending on context/dialect).
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6
> | Arabizi Input | Target Arabic | Meaning
> -- | -- | -- | --
...

The examples highlighted a bug in the mapping: `e` was incorrectly mapped to ع (ain), but `3` already covers ع and `e` is universally used as a vowel sound in Arabizi (e.g. "Keefak" → كيفك, "ta3mel" → تعمل). Fixed in b090c51:

- `e` → ي (ya — long 'ee' sound, was ع which was wrong)
- `u` → و (waw — was missing; fixes "shu" → شو, "Mnoo7" → منوح)

The short-vowel ambiguity (e.g. `a` producing an extra ا) is an inherent limitation of Arabizi — the function returns multiple candidates so the search engine can match any plausible interpretation.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6
>

> The Enhanced Spec (Logic Rules)
...

Implemented all four rules in commit `01c46a3`:

- **Rule A** (precedence): Already in place — digraphs are tried before single chars.
- **Rule B** (`el`/`al` prefix): Word-initial `el` or `al` now generates `ال` as an additional candidate. Spaces within a string reset the word-start flag so every word gets the same treatment. `"el 7al"` → includes `"ال حال"`.
- **Rule C** (gemination): Doubled non-vowel consonants generate `letter + ّ (shadda). `"ll"` → includes `"لّ"`, `"tasharrafna"` → includes candidate with `رّ`. Vowels (`a e i o u`) are excluded. Digraph units (e.g. `shsh`) are documented as producing two distinct letters, not a geminated one.
- **Rule D**:
  - `eh` → `ة` added to `ARABIZI_DIGRAPHS`; `"salameh"` → includes `"سالامة"`.
  - Terminal `a` → also `3"` ;`ى"la` → includes `"على"`.
  - Initial `a` → also `أ`; `"a5barak"` → includes `"أخباراك"`.

Security summary: CodeQL found 0 alerts. No security issues introduced.
</comment_old>

<comment_old>
<author>@assem-ch</author>
arabizi_to_arabic_list("salameh")   # [..., 'سلامة']   — Rule D: eh → ة
arabizi_to_arabic_list("a5barak")   # ['أخبارك' ,'اخبارك'] — Rule D: initial a → أ
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6
generate all potentials and test the existence in the list of unvocalized quran words
arabizi_to_arabic_list("salameh") # [..., 'سلامة'] — Rule D: eh → ة
arabizi_to_arabic_list("a5barak") # ['أخبارك' ,'اخبارك'] — Rule D: initial a → أ
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6
> generate all potentials and test the existence in the list of unvocalized quran words
> arabizi_to_arabic_list("salameh") # [..., 'سلامة'] — Rule D: eh → ة
...

Done in `0aa0cac`:

Short vowel omission — every vowel (`a`, `e`, `i`, `o`, `u`) now also generates an empty-string candidate, matching unvocalized Arabic where short vowels are not written:
- `arabizi_to_arabic_list("salameh")` → includes `'سلامة'`
- `arabizi_to_arabic_list("a5barak")` → includes `'اخبارك'`, `'أخبارك'`

Quran word filtering — added `quran_unvocalized_words()` in `data.py` (loads `word.json`, returns a cached frozenset of 14 870 unvocalized words) and `filter_candidates_by_wordset()` in `romanization.py` as the shared filtering utility. `outputs.py` (QSE) now filters each Arabizi query's candidates to those where every token is a known Quranic word, falling back to all candidates when none match (e.g. dialect words not in the Quran).

Example: `ya36eek` → filtered to `['يعطيك']`; `salameh` → no Quran match, falls back to all candidates.
</comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6 change test to "el 7md" == "الحمد"

>   assert arabizi_search("7md") > 0   # حمد (hamd/praise)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E   AssertionError: assert 0 > 0
E   + where 0 = </comment_old>

<comment_old>
<author>@assem-ch</author>
@copilot try again

</comment_old>

<comment_old>

<author>@copilot</author>

> @copilot try again
>

Done in 2f43c24. Changed the test from `arabizi_search("7md")` to `arabizi_search("el 7md")` — `"el 7md"` transliterates to `"الحمد"` (with the definite article), which appears in the Quran and returns results. `"حمد"` alone (without ال) doesn't match any unvocalized verse text.
</comment_old>

<comment_new>
<comment_id>3994023889</comment_id>
<author>@assem-ch</author>
@copilot+claude-sonnet-4.6 examples of quran Word with different arabizi styles,
Arabizi Style 1 (Phonetic Roman)
Arabizi Style 2 (Number-based Chat)
Arabizi Style 3 (Simplified Phonetic)
Arabizi Style 4 (Dialectal Variation)

Notice a char can be mapped to different chars, our objective is to list all potentials and filter them comparing to Quran words, if empty, keep one

1
الله
Allah
Allah
Allaah
Alla
2
الشيطان
Ash-shaytan
Ash-shay6an
Ash shaytan
El shitan
3
أنت
Anta
2anta
Anta
Enta
4
أنا
Ana
2ana

Ana
Ana
5
نحن
Nahnu
Na7nu
Nahno
Nahna
6
خير
Khayr
5ayr
Khair
Khir
7
اسم
Ism
2ism
Esm
Ism
8
الرحمن
Ar-Rahman
Ar-ra7man
Ar Rahman
Er rahman
9
الرحيم
Ar-Rahim
Ar-ra7im
Ar Rahim
Er rahim
10
الكريم
Al-Karim
Al-karim
Al Kareem
El karim
11
الحمد
Al-Hamd
Al-7amd
Al Hamd
El hamd
12
العالمين
Al-Alamin

Al-3alameen
Al Aalamien
El alamin
13
مسلم
Muslim
Muslim
Moslem
Moslim
14
مؤمن
Mu'min
Mu2min
Momin
Mo'men
15
مشرك
Mushrik
Mushrik
Mushrek
Mushrek
16
كافر
Kafir
Kafir
Kafer
Kafer
17
صالح
Salih
9ali7
Saleh
Saleh
18
يوم
Yawm
Yawm
Yom
Yom
19
الدين
Ad-Din
Ad-deen
Ad Din
Ed din
20
إياك

Iyyaka
2iyyaka
Eyak
Iyak
21
الصراط
As-Sirat
A9-9ira6
As Sirat
Es sirat
22
المستقيم
Al-Mustaqim
Al-musta9im
Al Mustaqeem
El mustaqim
23
الذين
Alladhina
Alladheen
Al ladheen
El ladhin
24
غير
Ghayr
3'ayr
Ghair
Ghayr
25
ضالين
Dallin
6alleen
Dalleen
Dallin
26
كتاب
Kitab
Kitab
Ketab
Ketab
27
أنزلنا
Anzalna
2anzalna
Anzalna
Anzalna
28

آية
Ayah
2ayah
Aya
Aya
29
الأرض
Al-Ard
Al-2ar6
Al Ard
El ard
30
السماء
As-Sama'
As-sama2
As Sama
Es sama
31
القرآن
Al-Qur'an
Al-9ur2an
Al Quran
El quran
32
عمل
Amal
3amal
Amal
Amal
33
في
Fi
Fi
Fee
Fy
34
قال
Qala
8ala
Qala
Ala
35
قالوا
Qalu
8aloo
Qaloo
Alu

```
36
كان
Kana
Kana
Kan
Kaan
37
كانوا
Kanu
Kanoo
Kanoo
Kanu
38
السموات
As-Samawat
As-samaawat
As Sam...
</comment_new>

</pr_comments>

</comments>

---
The last **7** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.